Draft version April 14, 2026
Typeset using LaTeX **preprint** style in AASTeX631

HW Set 05: FFT - PDE Solving

Kaitlyn Parrinello[1]

[1] *Homer L. Dodge Department of Physics and Astronomy, University of Oklahoma, Norman, OK 73019, USA*

ABSTRACT

The fifth homework: it's Fourier transform time. All code can be found in the Jupyter Notebook: https://github.com/kparrine/Computational_Astro/blob/main/C5/C5.ipynb.

1. QUESTION 1

The analytical computation can be found here: https://github.com/kparrine/Computational_Astro/blob/main/C5/Q1a.pdf.

Figure 1 is the discrete Fourier transform (DFT) for N = 50, 150, and 400 of

$$f(x) = e^{-50(x-\frac{1}{2})^2}$$

Also, the analytic DFT is plotted with N = 100. When zoomed in, as the values of N get smaller, the DFT looks more like a Gaussian. This is due to the property of a Fourier transform that transforming a Gaussian yields a Gaussian. To recover the shape of the original function, I invert the DFTs with the inverse Fourier transform (IDFT) in Figure 2. As expected, the original function is recovered in all three instances.

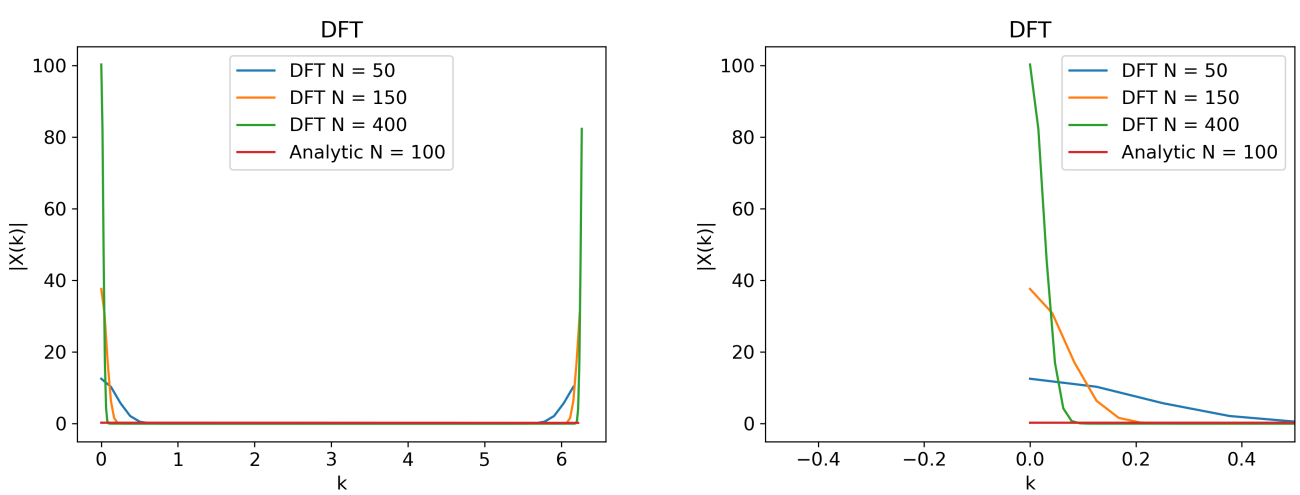

Figure 1. (a) DFT for N = 50 (blue), N = 150 (orange), N = 400 (green), and the analytical function (red), (b) zoomed in of the same plot.

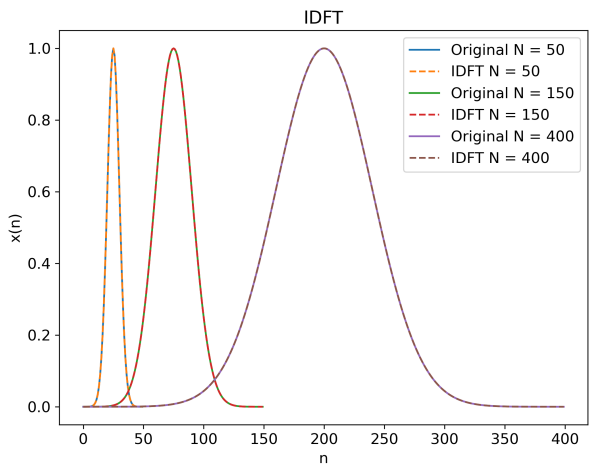

Figure 2. IDFT for N = 50, N = 150, and N = 400. The original functions are solid lines and the recovered functions from IDFT are dashed lines.

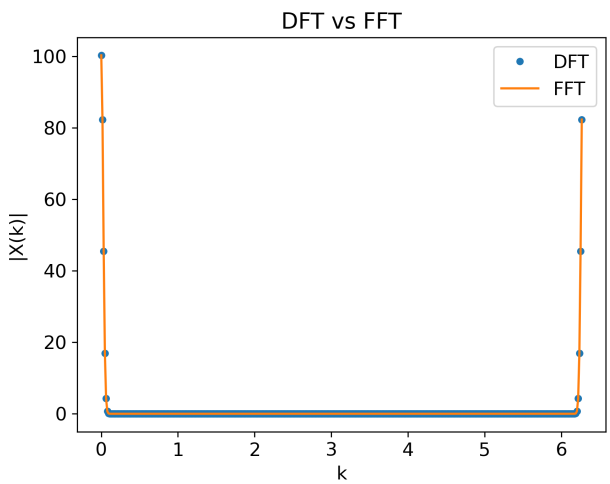

Figure 3. DFT vs FFT plot.

Figure 3 is comparing my DFT code with the fast Fourier transform (FFT) from Scipy with `scipy.fft`. Both functions perform the same transform, so the result should be the same, and indeed it is. However, `scipy.fft` works a lot faster and can handle larger values of N which is visualized in the time complexity plot for DFT vs FFT in Figure 4. As expected, the DFT is more computationally expensive than the FFT. The DFT grows as $\mathcal{O}(N^2)$, but looks linear in the log-log plot because it takes a longer time to go through the loops at larger values of N. The FFT grows much more slowly as $\mathcal{O}(NlogN)$, but still takes some time for high values of N, although not as much as DFT.

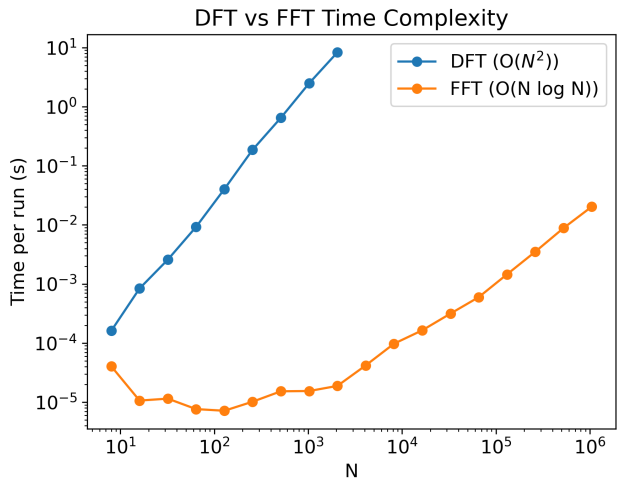

Figure 4. Time complexity plot.

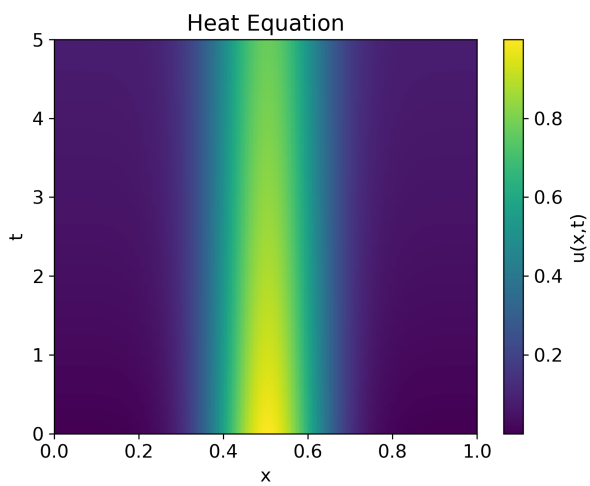

Figure 5. Solution to the heat equation with $\alpha = 0.005$ and time step $\Delta t = 0.001$.

2. QUESTION 2

I use my DFT code from Q1 to Fourier transform the initial heat distribution, then I use my Runge–Kutta 4th order numerical integrator to solve the ODE in k-space, and finally I use my inverse Fourier transform to go back to position space. With parameters $x \in [0, 1]$, $\alpha = 0.005$, time step $\Delta t = 0.001$ and total integration time T = 5, I plot the solution as a function of space and time with a color plot in Figure 5. There is a narrow and bright band centered at $x = 0.5$ where the value of $u(x, t)$ is highest. Even as time goes on, the narrow band stays, but the place where $u(x, t)$ is at a maximum becomes thinner and thinner. Any position from $x = 0.5$ either decreasing or increasing will decrease the value of $u(x, t)$ regardless of how much time has passed.

I also calculate the total heat at each time step in Figure 6. The total heat stays constant at each time step.

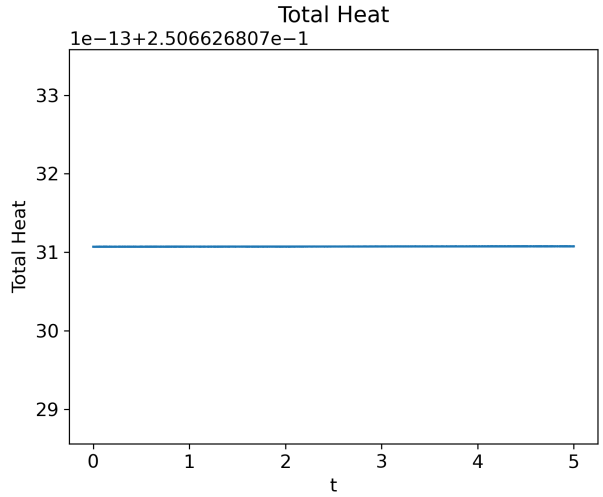

Figure 6. Total heat at each time step.

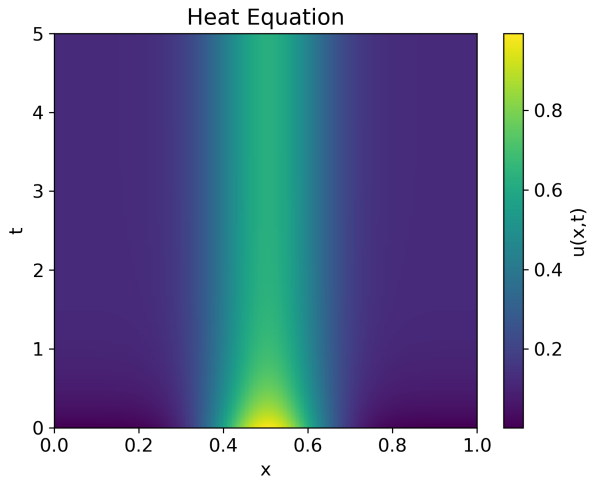

Figure 7. Solution to the heat equation with $\alpha = 0.05$ and time step $\Delta t = 0.01$.

Now I choose $\alpha = 0.05$, time step $\Delta t = 0.01$ with the result in Figure 7. The plot looks very similar to Figure 5, but it falls off a lot faster where the $u(x,t)$ is at a maximum only at $t = 0$. Numerically, a larger Δt means that some accuracy is lost, so perhaps the effect is that $u(x,t)$ falls off much faster since larger time steps are computed.